SA Time Stability Tools 0.0.3

Time Stability App – README

William W. Wallace NRAO / GBO wwallace@nrao.edu

Version 0.0.3 2026-10-11 GPL-3.0-or-later Internal draft

Table of contents

Overview 2
1 What changed in 0.0.3 2
2 Contents 3
3 Installation (mamba, conda-forge) 3
4 Running the GUI 3
4.1 Spyder . . . 4
5 Canonical CSV format 4
6 Vendor spectrum-analyzer conversion — important caveat 4
7 Stable32 export 4
8 Veusz export (`.vszh5`) 5
8.1 Rendering (optional) . . . 5
9 Performance and parallelism 5
10 Time-domain noise identification 6
11 Frequency-domain (SSB phase noise) identification 6
12 Equation appendix 7
12.1 Overlapping Allan deviation (OADEV) . . . 7
12.2 Modified Allan deviation (MDEV) . . . 7
12.3 Time deviation (TDEV) . . . 7
12.4 Overlapping Hadamard deviation (OHDEV) . . . 7
12.5 Thêo1 . . . 7
12.6 ThêoBR and ThêoH . . . 7
12.7 SSB phase noise . . . 8
12.8 Fractional frequency from phase, and phase from frequency . . . 8
12.9 Polynomial drift removal . . . 8
12.10 Linear frequency drift in the Allan deviation . . . 8
13 Validation performed (0.0.3) 8
14 Notes for the shared writer `vszh5_writer.py` 0.0.5 9
15 Open items 9
16 IEEE reference table 9

Overview

Python toolkit and desktop GUI for long-time-measurement frequency stability and phase noise. It computes NIST SP 1065 time- and frequency-domain statistics [1], derives SSB phase noise, identifies power-law noise types and frequency drift, converts vendor spectrum-analyzer exports, and writes Stable32 files and native Veusz HDF5 (`.vszh5`) documents.

Version	**0.0.3** (Semantic Versioning 2.0.0 [27]; `0.0.x` = internal draft)
Date	2026-10-11
Author	William W. Wallace · NRAO/GBO · wwallace@nrao.edu
Licence	GPL-3.0-or-later (SPDX); full text in `LICENSE`
GUI	`Time_Stability_App.py` (Qt.py → PySide6)
Python	3.8 – 3.13. Suite run on 3.8.20, 3.12.13 and 3.13.16
Veusz	Not needed to write the `.vszh5`; checked against Veusz 3.6.2 and 4.2.1
Status	Internal draft, validated on synthetic data only

0.0.3 is pass 2 of 2. Pass 1 (0.0.2) moved the Veusz output to the native HDF5 writer and renamed the GUI. Pass 2 fixes everything found in review, parallelises the analysis and re-checks every reference. The full history is in `CHANGELOG.md`.

1 What changed in 0.0.3

#	Problem (0.0.1/0.0.2)	Fix in 0.0.3	Checked by
1	A `freq_hz`/`frac_freq` CSV loaded with `len(x) = len(t)+1`; drift removal, plots and export failed	Time tags extended by one interval; `load_canonical_series()` reports the column read	`validate_core` T9, `validate_gui`
2	Loading a new file kept the old drift fit and results (stale residual applied)	Results cleared on load; late results of a running job discarded	`validate_gui`
3	GUI always logged "converting to phase"; README said the loader returned the column type	Log names the column; f0 filled from `freq_hz` data	`validate_gui`
4	Decimal-comma exports (`;`, tab) read as "no numeric data"	Delimiter chosen from the data lines; decimal commas read	`validate_sa_convert`
5	Converted CSV kept 11 significant digits (10 GHz series: 3.7 mHz error on 1.2 mHz steps)	Shortest exact decimal (bit-exact round trip)	`validate_sa_convert`
6	Stable32 writer left a partial file when it rejected a last-point gap	Validation before writing; atomic `.part` + rename	`validate_stable32_export`
7	`NOISE_TABLE` "Linear Freq. Drift" +1.5 was Random-Run FM; drift (+1) reported as Flicker-Walk FM	Row renamed; drift detected from OADEV +1 with a flat/falling OHDEV	`validate_core` T10
8	Drift rate logged as "s/s^2"	$D = 2a_2$ in 1/s	`validate_core` T11
9	Everything single-threaded on the GUI thread; ThêoH of 10^5 points took 218 s	Worker threads in the GUI; faster Thêo1 kernel; thread/process pools; optional CuPy	`validate_core` T8, below
10	References garbled (wrong DOI, pages, authors, merged citations)	All checked against primary records; table below	—
11	Three validation scripts always exited 0	Every script returns PASS/FAIL; `run_all.py`	`run_all.py`
12	`PySide6~=6.6.3` pin: no Python 3.13 wheels	Version-gated pins	install
13	GUI: no Qt.py, no menus, no light/dark display	Qt.py; File/Analysis/Export/View/Help; View › Light/Dark (remembered)	`validate_gui`
—	Phase-noise ID from two-bin differences named the right type in 67 % of segments	Windowed least-squares slope: 99 %	`validate_phase_noise`
—	ThêoH stopped at 62.8 % of the record	Grid ends at the largest even $m \le N - 1$: 75 %	`validate_core` T6
—	Long records drawn point by point	Min/max envelope for display (GUI and Veusz pages); every sample analysed and stored	`validate_veusz_export`

2 Contents

File	Version	Purpose
`Time_Stability_App.py`	0.0.3	Qt GUI (Qt.py → PySide6)
`stability_core.py`	0.0.3	CSV I/O, drift fit, ADEV/OADEV/MDEV/TDEV/OHDEV/Thêo1/ThêoBR/ThêoH, noise and drift identification, thread/process/GPU engine
`phase_noise.py`	0.0.3	SSB L(f) via Welch PSD, direct traces, frequency-domain noise ID, L(f)↔S_y(f)
`sa_convert.py`	0.0.3	Keysight / R&S / generic exports → canonical CSV; parallel batch marker extraction
`stable32_export.py`	0.0.3	Stable32 phase, frequency and sigma-tau files
`veusz_export.py`	0.0.3	Native Veusz HDF5 export, display envelope, optional render
`vszh5_writer.py`	0.0.5	Shared Veusz HDF5 writer, unchanged and byte-identical to QuickSyn 0.0.5 (SHA-256 `2cdf43d2…ae33c1c`); see Notes for the shared writer
`validate_core.py`	0.0.3	11 tests: estimators vs allantools, slopes, Thêo1, ThêoH grid, parallel = serial, CSV loader, drift
`validate_phase_noise.py`	0.0.3	L(f) slopes (independent seeds), noise-ID accuracy, S_y(f)
`validate_sa_convert.py`	0.0.3	Vendor files incl. decimal commas, precision, parallel batch
`validate_stable32_export.py`	0.0.3	New: format, gaps, atomic write, sigma-tau table
`validate_veusz_export.py`	0.0.3	`.vszh5` layout, bit-exact data, envelope — no Veusz needed
`validate_vszh5_in_veusz.py`	0.0.3	Round trip through Veusz's own loader and renderer (needs Veusz)
`validate_gui.py`	0.0.3	Headless GUI test: menus, themes, worker threads, exports
`run_all.py`	0.0.3	New: runs every `validate_*.py` in parallel, summary table, exit status
`environment.yml` / `requirements.txt`	0.0.3	mamba (conda-forge) environment / pip alternative
`README.md`, `CHANGELOG.md`, `LICENSE`	0.0.3	Documentation (Markdown with $…$ LaTeX for Obsidian); GPL-3.0 text
`README.pdf`	0.0.3	This README printed from HTML (Chromium); PDF bookmarks follow the headings
`README_LuaLaTeX.pdf`, `latex/README_LuaLaTeX.tex`	0.0.3	The README typeset with LuaLaTeX: numbered sections, contents, bookmark hierarchy, linked references. Rebuild with `latexmk -lualatex README_LuaLaTeX.tex`
`example_data/example_maser_phase.csv`	—	Example record (8193 points, `time_s,phase_s`). Synthetic in scale (x of order 10^3 s), so L(f) lies far above 0 dBc/Hz; for workflow checks only

3 Installation (mamba, conda-forge)

```
mamba env create -f environment.yml
mamba activate sa_stability
python Time_Stability_App.py
python run_all.py --veusz-python python      # optional: the validation suite
```

`h5py` writes the `.vszh5`; Veusz is in the environment only to open the documents and render PDF/PNG (remove it if you use the Veusz Windows installer, then set `TIME_STABILITY_VEUSZ` to `veusz.exe`). `threadpoolctl` lets the Thêo1 thread pool cap BLAS threads (without it a BLAS-free kernel is used). For the GPU path add `cupy` (or `cupy-cuda12x` with pip) to match your CUDA driver.

pip alternative: `python -m pip install -r requirements.txt`.

4 Running the GUI

Menu	Items (shortcut)
File	Load Canonical CSV (Ctrl+O), Convert Vendor SA File, Close Data — frees memory (Ctrl+W), Quit (Ctrl+Q)
Analysis	Remove Drift (Ctrl+D), Run Stability Analysis (Ctrl+T), Run SSB Phase Noise (Ctrl+P), Run All (Ctrl+R)
Export	Stable32 file — phase `.PHD`, frequency `.FRD` or sigma-tau table `.DAT`; Veusz HDF5 `.vszh5` (Ctrl+E); Render a `.vszh5` to PDF/PNG
View	Light display / Dark display (Fusion palette and plot colours; remembered)
Help	About (versions, Qt binding, CPUs, GPU)

Workflow: load a canonical CSV (or convert an analyzer export and say whether the series holds `phase_s`, `freq_hz` or `frac_freq`) → check `tau0` (from the time stamps) and `f0` (filled from `freq_hz` data) → Run All, or drift → stability → phase noise → export. Analyses run on a worker thread: the window stays live, buttons are disabled while a job runs, and the status bar shows the job, the thread count and the GPU state. **Threads** sets the workers of the analysis engine (default: all CPUs); **GPU (CuPy)** is enabled when CuPy is installed.

4.1 Spyder

Every file is split into `# %%` cells (header, imports, one cell per section, entry point, a commented scratch cell) and runs cell by cell in order — checked with a harness that reproduces Spyder's `runcell` namespace rules (`__file__` defined only while a cell runs). Scripts capture their path once, never call `sys.exit()` inside IPython, and the GUI reuses an existing `QApplication`. With the console's graphics backend set to Qt the window stays live and `main()` returns at once; with the inline backend `main()` runs the event loop until the window closes. The process pools never re-run your script or console cell in their workers (see Performance).

5 Canonical CSV format

```
time_s,phase_s        # phase / time-error data (seconds), OR
time_s,freq_hz        # absolute frequency readings (Hz), OR
time_s,frac_freq      # fractional frequency y = (f - f0)/f0
```

`stability_core.load_canonical_series()` reads the file with NumPy's fast parser (a UTF-8 byte-order mark, `#` comments and any column order are accepted), sorts by time and returns the phase, the time tags (same length), the column read, `tau0` (median spacing) and the `f0` used. Frequency columns are integrated to phase (SP 1065 §4.1): N readings give N + 1 phase points, and reading k is taken to cover $[t_k, t_k + \tau_0]$. `freq_hz` is normalised by its median unless `f0_hz` is given; a constant fractional offset changes no AVAR-family statistic. `load_canonical_csv()` keeps its 0.0.1 return form `(t, x, 'phase')`.

6 Vendor spectrum-analyzer conversion — important caveat

Exact SA export layouts vary by instrument model and firmware. Export one small file from your instrument and confirm the units before trusting a bulk conversion. `sa_convert.py` provides:

- a generic parser for comma, semicolon, tab or white-space files with metadata/comment lines; the delimiter is chosen from the data lines, and with `;`, tab or white space a decimal comma (`1,234`) is read as 1.234 (with a comma delimiter a decimal comma is ambiguous and not supported);
- Keysight- and R&S-flavoured parsers that take the numeric block after the instrument's header (trailing `;` allowed); header words such as ”phase noise” or ”dBc” mark a phase-noise trace;
- time-series vs trace classification of column 0 (uniform vs log-uniform spacing);
- output at full double precision (shortest exact decimal); the value column of a time series must be named by the caller (`phase_s`, `freq_hz`, `frac_freq`) because the file does not say what the analyzer logged;
- `batch_extract_marker_time_series()` for repeated snapshots (peak or n-th largest value per file), parsed in a process pool for 8 or more files.

7 Stable32 export

Tab-delimited ASCII: `timetag value` (or the value column only with `timetags=False`), 16 significant digits, ”gap = 0” with a gap count in the # header, no gap allowed at the first or last point — checked **before** anything is written, and the file is written to `<name>.part` and renamed, so an error never leaves a truncated file [14]. The GUI exports the (drift-removed) phase `.PHD`, its fractional frequency `.FRD` or the sigma-tau table `.DAT` (τ, m, n, ADEV, OADEV, MDEV, TDEV, OHDEV; rows with a non-finite value are skipped and counted).

> **Data gap — time tags.** The time-tag column is elapsed seconds (as in 0.0.1). Whether Stable32 reads numeric time tags as MJD days when it derives τ from them could not be confirmed from the documentation available here. Enter τ in Stable32 yourself, or export without time tags. Reading the files in Stable32 itself has not been tested.

8 Veusz export (`.vszh5`)

`veusz_export.export_vszh5()` writes one native Veusz HDF5 document with h5py through `vszh5_writer.py` [17] — no Veusz and no Qt are imported, so the GUI writes it in-process. Every result is stored at float64; the write is atomic.

Dataset	Content	Tag
`raw/time_s`, `raw/phase_s`, `raw/residual_s`	Input and drift-removed x(t), every sample	raw
`raw/freq_time_s`, `raw/frac_freq`, `raw/frac_freq_residual`	$y_k = (x_{k+1} - x_k)/\tau_0$ at interval midpoints	raw
`plot/<series>_time_s`, `plot/<series>`	Min/max display envelope when a series is longer than 200 000 points	plot
`stability/tau_s, m, n_used, adev, oadev, mdev, ohdev, tdev_s`	AVAR-family sweep	stability
`stability/theoh_tau_s, theoh_dev, theoh_m, theoh_source`	ThêoH curve and the estimator of each point	stability
`noise_id/*` (incl. `slope_hdev`), `phase_noise_id/*`	Noise-identification tables	noise_id
`phase_noise/freq_hz`, `phase_noise/l_dbc_hz`	SSB L(f)	phase_noise
`curves/*`	Extra curves passed as `sigma_tau_series` (0.0.1 API)	curves
`meta/title, source, generator, parameters_json, datasets`	Run description, tau0, f0, drift fit, envelope flag	meta

Pages: `page_overview` (2 × 2), `page_phase`, `page_frequency`, `page_sigma_tau` (ADEV, OADEV, MDEV, OHDEV, ThêoH), `page_tdev` (seconds, on its own axes), `page_phase_noise`. Long series are drawn from the envelope (each bin keeps its minimum and maximum, in time order, so no excursion is lost); Veusz stays responsive with a few hundred thousand points per trace.

8.1 Rendering (optional)

`render_vszh5()` / `python veusz_export.py --render FILE.vszh5` writes one PDF of all pages and one PNG per page. The Veusz program is driven in `--listen` mode with its HDF5 loader (`veusz --export` cannot open a `.vszh5`), with the veusz Python package as fallback; the program is searched in `TIME_STABILITY_VEUSZ`, `VEUSZ_EXE`, `PATH`. The GUI renders in a child process, so Veusz's Qt never loads next to PySide6.

9 Performance and parallelism

Stage	How it runs
GUI	Analyses on a worker thread; results delivered by a Qt signal; stale results dropped
Thêo1 kernel	Array views and two reused buffers per averaging factor — no temporaries, no index arrays
ThêoH	Every Thêo1 the curve needs (bias ladder + grid) in one batch, largest first: a spawn **process pool** for ≥ 100 000 points, a **thread pool** for ≥ 50 000, serial below (measured: threads lose there)
One large Thêo1	Split over the w-range on threads
BLAS	Capped to one thread inside the pools (threadpoolctl). Uncapped, OpenBLAS's own threads oversubscribe the CPUs: one Thêo1 point (N = 100 000, m = 20 000) took 3.89 s on 2 threads vs 1.48 s serial and 0.95 s on 2 threads with the cap
AVAR sweep	One thread per averaging factor for long records
GPU	`use_gpu=True` / GUI **GPU (CuPy)**: Thêo1 sums with CuPy
Welch FFT	`scipy.fft` worker threads
Batch conversion	Process pool (thread fallback)
Display	Min/max envelope (50 000 points on screen, 200 000 in Veusz)

Measured on the 2-vCPU test VM (whose ceiling for perfectly parallel NumPy work is 1.47×), white/flicker FM, N = 100 000:

Quantity	0.0.2	0.0.3 serial	0.0.3 parallel
Thêo1, m = 2 000	1.26 s	0.23 s	0.15 s (2 threads)
Thêo1, m = 20 000	8.31 s	1.75 s	1.07 s (2 threads)
Thêo1, m = 50 000	11.84 s	2.64 s	1.79 s (2 threads)
Full ThêoH curve	218.1 s	26.5 s	23.0 s (2 processes)

Results are identical to the serial path to rounding ($\leq 2 \times 10^{-15}$) and to 0.0.2 (Thêo1 agrees with `allantools.theo1` to $\leq 6.6 \times 10^{-16}$ on the benchmark records, 2.5×10^{-16} in `validate_core`). Larger gains need more cores; they were not measurable here. The GPU path is written against CuPy's NumPy API and has **not** been run (no GPU here).

Spawned workers normally re-execute the parent's main script; while a pool starts, `__main__`'s `__file__`/`__spec__` are hidden from multiprocessing, so a cell-style script, a Spyder console or the GUI is not re-run (or re-imported with Qt) in each worker [28].

10 Time-domain noise identification

$\sigma_y^2(\tau) \propto \tau^{\mu}$ with $S_y(f) \propto f^{\alpha}$ and $\mu = -\alpha - 1$ for $-4 \leq \alpha < 1$ ($\mu \approx -2$ for $\alpha \geq 1$); the ADEV log-log slope is $\mu/2$ [1], [3]:

Noise type	α	ADEV slope	MDEV slope
White PM	2	−1.0	−1.5
Flicker PM	1	−1.0	−1.0
White FM	0	−0.5	−0.5
Flicker FM	−1	0.0	0.0
Random Walk FM	−2	+0.5	+0.5
Flicker Walk FM	−3	+1.0 †	+1.0 †
Random Run FM	−4	+1.5 †	+1.5 †
Linear frequency drift	—	+1.0	+1.0

† AVAR and MVAR converge only for $\alpha > -3$; these rows are the Hadamard (HDEV) slopes, which converge for $\alpha > -5$. 0.0.1/0.0.2 labelled the +1.5 row "Linear Freq. Drift". A drift D gives $\sigma_y(\tau) = D\tau/\sqrt{2}$ (slope +1, the same as Flicker-Walk FM), while the Hadamard deviation does not see it: `classify_noise()` reports **Linear Freq. Drift** when the OADEV slope is within 0.3 of +1 and the OHDEV slope is at least 0.5 lower (or OHDEV is zero). White PM vs Flicker PM is separated by the MDEV slope (−1.5 vs −1.0).

11 Frequency-domain (SSB phase noise) identification

Noise type	L(f) slope (dB/decade)
White PM	0
Flicker PM	−10
White FM	−20
Flicker FM	−30
Random Walk FM	−40

$\mathcal{L}(f) = 10\log_{10}[\frac{1}{2}S_\phi(f)]$ is already in dB, so its slope against $\log_{10} f$ is 10× the S_ϕ exponent, with $S_\phi = (f_0/f)^2 S_y$ [3]. L(f) is averaged in log-spaced bins; from 0.0.3 each segment's slope is a least-squares fit over ±0.25 decade of bins. On 15 synthetic records (5 noise types × 3 seeds, 0.05–20 Hz) the 0.0.2 two-bin method named the injected type in 47–84 % of segments (mean 67 %), the windowed fit in 95–100 % (mean 99 %). With independent seeds the measured slopes are −19.80 / −29.88 / −40.06 dB/decade for white / flicker / random-walk FM; the −0.8 dB/decade offset reported for 0.0.1 came from one random realisation shared by all three cases.

12 Equation appendix

Equations use $$ … $$ and $ … $ so they render in Obsidian. x_i are phase samples at spacing τ_0, N their number, $\tau = m\tau_0$.

12.1 Overlapping Allan deviation (OADEV)

SP 1065 [1] Eq. (11):

$$\sigma_y^2(m\tau_0) = \frac{1}{2(m\tau_0)^2(N-2m)} \sum_{i=1}^{N-2m} (x_{i+2m} - 2x_{i+m} + x_i)^2$$

12.2 Modified Allan deviation (MDEV)

SP 1065 [1] Eq. (14):

$$\mathrm{Mod}\,\sigma_y^2(m\tau_0) = \frac{1}{2m^2(m\tau_0)^2(N-3m+1)} \sum_{j=1}^{N-3m+1} \left[\sum_{i=j}^{j+m-1} (x_{i+2m} - 2x_{i+m} + x_i) \right]^2$$

12.3 Time deviation (TDEV)

SP 1065 [1] Eq. (15):

$$\sigma_x(\tau) = \frac{\tau}{\sqrt{3}}\,\mathrm{Mod}\,\sigma_y(\tau)$$

12.4 Overlapping Hadamard deviation (OHDEV)

SP 1065 [1] Eq. (20):

$$\sigma_H^2(m\tau_0) = \frac{1}{6(m\tau_0)^2(N-3m)} \sum_{i=1}^{N-3m} (x_{i+3m} - 3x_{i+2m} + 3x_{i+m} - x_i)^2$$

12.5 Thêo1

SP 1065 [1] Eq. (30) and [5]; m even, $10 \le m \le N-1$. SP 1065 sums over $\delta = 0 \ldots m/2 - 1$; with $w = m/2 - \delta$ and a 0-based i the sum reads:

$$\mathrm{Theo1}(m, \tau_0, N) = \frac{1}{0.75\,(N-m)\,(m\tau_0)^2} \sum_{i=0}^{N-m-1} \sum_{w=1}^{m/2} \frac{[(x_i - x_{i+w}) + (x_{i+m} - x_{i+m-w})]^2}{w}$$

reported at the stride ("Theo1 averages every permissible squared second-difference … in a given span or stride $\tau_s = 0.75\,m\,\tau_0$" [5]):

$$\tau = 0.75\,m\,\tau_0$$

12.6 ThêoBR and ThêoH

Howe [4] Eqs. (6)–(7):

$$G = \frac{1}{n+1} \sum_{i=0}^{n} \frac{\mathrm{AVAR}(m = 9 + 3i)}{\mathrm{Theo1}(m = 12 + 4i)}, \qquad n = \left\lfloor \frac{0.1N}{3} - 3 \right\rfloor, \qquad \mathrm{ThêoBR}(m) = \sqrt{G} \cdot \mathrm{Theo1}(m)$$

$$\mathrm{ThêoH} = \begin{cases} \mathrm{AVAR}(m), & 1 \le m < k/\tau_0 \\ \mathrm{ThêoBR}(m), & k/(0.75\,\tau_0) \le m \le N-1,\ m \text{ even} \end{cases}$$

with k the largest $\tau \le 10$ % of the record. The code subsamples the ladder to at most 40 evenly spaced rungs and grows the ThêoBR grid by 1.3× per step, ending at the largest even $m \le N - 1$ (75 % of the record).

12.7 SSB phase noise

IEEE Std 1139 [3]:

$$\phi(t) = 2\pi f_0\, x(t), \qquad \mathcal{L}(f) = 10 \log_{10}\!\left[\tfrac{1}{2}\, S_\phi(f)\right] \text{ dBc/Hz}$$

with $S_\phi(f)$ the one-sided Welch [11] PSD of $\phi(t)$, and [3]

$$S_\phi(f) = 2 \cdot 10^{\mathcal{L}(f)/10}, \qquad S_y(f) = \frac{f^2}{f_0^2}\, S_\phi(f)$$

12.8 Fractional frequency from phase, and phase from frequency

SP 1065 [1] §4.1:

$$y_k = \frac{x_{k+1} - x_k}{\tau_0}, \qquad x_0 = 0, \quad x_{k+1} = x_k + y_k\, \tau_0$$

12.9 Polynomial drift removal

Least squares on a time axis mapped to $[-1, 1]$; the residual $\varepsilon(t)$ feeds every estimator:

$$x(t) = a_0 + a_1 t + a_2 t^2 + \varepsilon(t), \qquad D = \frac{dy}{dt} = 2a_2 \quad [\mathrm{s}^{-1}]$$

12.10 Linear frequency drift in the Allan deviation

For $y(t) = Dt$:

$$\sigma_y(\tau) = \frac{D\,\tau}{\sqrt{2}}, \qquad \sigma_H(\tau) = 0$$

13 Validation performed (0.0.3)

Run with `run_all.py` on Python 3.8.20 (NumPy 1.24.4, SciPy 1.10.1, PySide6 6.6.3.1), 3.12.13 and 3.13.16 (NumPy 2.5, PySide6 6.12), Qt.py 2.0.7; Veusz 3.6.2 (Ubuntu) and 4.2.1 (PyPI source build).

Script	Checks	Result
`validate_core.py`	11 tests, 35 checks: OADEV/MDEV/OHDEV/TDEV = allantools (0.000000 %); Thêo1 = `allantools.theo1` ($\le 2.5 \times 10^{-16}$); Thêo1/OADEV(τ) = 1.000…0.971; ThêoH grid per Howe, 75.0 % coverage; parallel = serial; BLAS-free kernel; frequency CSVs; drift detection; drift fit	PASS
`validate_phase_noise.py`	Slopes −19.80/−29.88/−40.06 dB/decade; noise ID 283/285 = 99 %; S_y formula	PASS
`validate_sa_convert.py`	10 checks incl. decimal-comma R&S file, bit-exact 10 GHz round trip, 12-file parallel batch	PASS
`validate_stable32_export.py`	16 checks: read-back, gaps, no partial/stale files, table	PASS
`validate_veusz_export.py`	26 checks incl. 300 000-point envelope and `slope_hdev`	PASS
`validate_vszh5_in_veusz.py`	22/22 with Veusz 3.6.2 and 4.2.1 (loader, bit-exact, tags, references, renders; 250 000-point document)	PASS
`validate_gui.py`	26 checks: menus, light/dark, worker thread, stale discard, reload, `freq_hz`, all exports, render, decimal-comma trace, Close Data	PASS

Script	Checks	Result
Spyder cells	Every `.py` file run cell by cell under Spyder's `runcell` rules	PASS

Not verified: Windows (paths, the Veusz installer in `--listen` mode, spawn pools from a frozen app), a real `mamba env create`, the GPU path, instrument data, Stable32 reading the files, machines with more than 2 CPUs.

14 Notes for the shared writer `vszh5_writer.py` 0.0.5

Found while testing this package; the shared module is unchanged here and `veusz_export.render_vszh5()` works around all four. Worth fixing in the next writer release (QuickSyn):

1. `render_pages(png_pages="first")` writes **no PNG with Veusz 4.2.1**: the listen script limits the loop with `min(1, len(names))`, and Veusz 4.2.1 evaluates scripts with NumPy's names imported, so `min` is `numpy.min(1, axis=n)` → `AxisError`; `--listen` prints the error and still prints the done marker, so the call reports success. `(1 if names else 0)` avoids it. QuickSyn's export default is `--png-pages first`.
2. The PDF is named `stem.with_suffix(".pdf")`, so `run v0.0.2.vszh5` gives `run v0.0.pdf`; use `stem.parent / (stem.name + ".pdf")`.
3. The returned list holds only files that did not exist before the call, so a re-render returns `[]`.
4. `find_veusz_exe()`/messages hard-code `QUICKSYN_VEUSZ`; `render_pages()` has no `env_vars` parameter. Minor: `local_utc_offsets()` assumes offsets change on whole UTC hours, not true for half-hour zones.

15 Open items

1. Confirm GPL-3.0-or-later for this package (needed because it ships the GPL `vszh5_writer.py`) and NRAO/GBO release approval.
2. Stable32 time-tag convention (data gap above); test the files in Stable32.
3. Windows test: environment, GUI, render, process pools.
4. A GPU run of the CuPy path.
5. Keysight and R&S export samples from your instruments for the converter.

16 IEEE reference table

”Checked” = how the entry was verified in this revision.

#	Reference	URL	Checked
[1]	W. J. Riley, ”Handbook of Frequency Stability Analysis,” NIST Special Publication 1065, Jul. 2008.	https://tf.nist.gov/general/pdf/2220.pdf	Document read (sections, equation numbers)
[2]	G. Kamas and M. A. Lombardi, ”Time and Frequency Users Manual,” NIST Special Publication 559 (Rev. 1990), Sep. 1990.	https://tf.nist.gov/general/pdf/461.pdf	First page read. 0.0.1 merged it with [10]
[3]	*IEEE Standard Definitions of Physical Quantities for Fundamental Frequency and Time Metrology — Random Instabilities*, IEEE Std 1139-2008, 2009, doi: 10.1109/IEEESTD.2008.4797525.	https://ieeexplore.ieee.org/document/4797525	DOI matches the Xplore record
[4]	D. A. Howe, ”ThêoH: a hybrid, high-confidence statistic that improves on the Allan deviation,” *Metrologia*, vol. 43, no. 4, pp. S322–S331, 2006, doi: 10.1088/0026-1394/43/4/S20.	https://doi.org/10.1088/0026-1394/43/4/S20	Crossref record and paper. 0.0.1: DOI …S17, pp. S322–S327
[5]	D. A. Howe and T. N. Tasset, ”Theo1: characterization of very long-term frequency stability,” in *Proc. 18th European Frequency and Time Forum*, 2004, pp. 725–728.	https://tf.nist.gov/general/pdf/1990.pdf	NIST record and paper (τ = 0.75 m τ_0). 0.0.1 named ”F. Vernotte”

#	Reference	URL	Checked
[6]	D. A. Howe, J. McGee-Taylor and T. Tasset, ”ThêoH bias-removal method,” in *Proc. 2006 IEEE Int. Frequency Control Symp.*, Miami, FL, USA, Jun. 2006, pp. 788–792.	https://www.nist.gov/publications/theoh-bias-removal-method	NIST record. 0.0.1 cited ”Purdue Oxide-MEMS Lab”
[7]	C. A. Greenhall and W. J. Riley, ”Uncertainty of stability variances based on finite differences,” in *Proc. 35th Annu. Precise Time and Time Interval (PTTI) Meeting*, San Diego, CA, USA, Dec. 2003.	https://ntrs.nasa.gov/citations/20050061319	NASA NTRS record; allantools refs. 0.0.1: Riley first, ”36th”
[8]	D. W. Allan, ”Statistics of atomic frequency standards,” *Proc. IEEE*, vol. 54, no. 2, pp. 221–230, Feb. 1966, doi: 10.1109/PROC.1966.4634.	https://doi.org/10.1109/PROC.1966.4634	As received (standard citation)
[9]	J. A. Barnes *et al.*, ”Characterization of frequency stability,” *IEEE Trans. Instrum. Meas.*, vol. IM-20, no. 2, pp. 105–120, May 1971; also NBS Technical Note 394, Oct. 1970.	https://nvlpubs.nist.gov/nistpubs/Legacy/TN/nbstechnicalnote394.pdf	TN 394 location found; IEEE DOI not checked
[10]	D. B. Sullivan, D. W. Allan, D. A. Howe and F. L. Walls, Eds., ”Characterization of Clocks and Oscillators,” NIST Technical Note 1337, Mar. 1990.	https://nvlpubs.nist.gov/nistpubs/Legacy/TN/nbstechnicalnote1337.pdf	Location found in a search; title the 0.0.1 README attached to SP 559
[11]	P. Welch, ”The use of fast Fourier transform for the estimation of power spectra: a method based on time averaging over short, modified periodograms,” *IEEE Trans. Audio Electroacoust.*, vol. 15, no. 2, pp. 70–73, Jun. 1967, doi: 10.1109/TAU.1967.1161901.	https://doi.org/10.1109/TAU.1967.1161901	As received; title completed
[12]	E. Rubiola, *Phase Noise and Frequency Stability in Oscillators*. Cambridge, U.K.: Cambridge Univ. Press, 2008, doi: 10.1017/CBO9780511812798.	https://doi.org/10.1017/CBO9780511812798	As received
[13]	A. Wallin *et al.*, ”allantools,” documentation and source (commit ddc5bb5, May 2026).	https://github.com/aewallin/allantools	Source read; used as reference implementation
[14]	W. J. Riley, ”Stable32 User Manual” and ”Stable32 Tour,” Hamilton Technical Services; Stable32 is distributed by the IEEE UFFC Society.	https://ieee-uffc.org/technical-committees/frequency-control/educational-resources/frequency-control-software ; https://wriley.com/Tour004.htm	Pages found; the 0.0.1 URL `stable32.com/Manual154.pdf` not verified
[15]	Keysight Technologies, ”N9068C Phase Noise Measurement Application — Measurement Guide” (X-Series).	https://www.keysight.com/	Not verified (document number)
[16]	Rohde & Schwarz, ”R&S FSWP Phase Noise Analyzer and VCO Tester — User Manual.”	https://www.rohde-schwarz.com/	Not verified (document number)
[17]	W. W. Wallace, `vszh5_writer.py` 0.0.5 (from `isp_speed_monitor.py` 1.3.0, Adaptive Electrodynamics LLC dba AdaptEd; shipped with QuickSyn 0.0.5), 2026.	Supplied by the author (not public)	SHA-256 matches the QuickSyn record
[18]	Veusz Developers, ”Veusz — a scientific plotting package,” and Veusz 4.2.1 source (`document/doc.py`, `document/loader.py`, `document/commandinterface.py`, `windows/mainwindow.py`).	https://veusz.github.io/ ; https://pypi.org/project/veusz/4.2.1/	Source read; 3.6.2 and 4.2.1 run
[19]	A. Collette *et al.*, ”h5py.”	https://pypi.org/project/h5py/	Package used (3.10, 3.11, 3.16)
[20]	M. Ottosson *et al.*, ”Qt.py — minimal Python 2 & 3 shim around all Qt bindings.”	https://github.com/mottosso/Qt.py	Package used (2.0.7)
[21]	The Qt Company, ”Qt for Python (PySide6) documentation.”	https://doc.qt.io/qtforpython-6/	Package used (6.6.3.1, 6.12)
[22]	J. D. Hunter, ”Matplotlib: a 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.	https://doi.org/10.1109/MCSE.2007.55	As received
[23]	P. Virtanen *et al.*, ”SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.	https://doi.org/10.1038/s41592-019-0686-2	Standard citation
[24]	C. R. Harris *et al.*, ”Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.	https://doi.org/10.1038/s41586-020-2649-2	Standard citation
[25]	R. Okuta, Y. Unno, D. Nishino, S. Hido and C. Loomis, ”CuPy: a NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on ML Systems (LearningSys)*, NIPS 2017.	https://learningsys.org/nips17/assets/papers/paper_16.pdf	Standard citation

#	Reference	URL	Checked
[26]	joblib developers, ”threadpoolctl.”	https://github.com/joblib/threadpoolctl	Package used
[27]	T. Preston-Werner, ”Semantic Versioning 2.0.0.”	https://semver.org/spec/v2.0.0.html	—
[28]	Python Software Foundation, ”multiprocessing — The spawn and forkserver start methods”; Spyder developers, `spyder-kernels` 3.1.6 source (`customize/namespace_manager.py`).	https://docs.python.org/3/library/multiprocessing.html ; https://github.com/spyder-ide/spyder-kernels	Source read

Removed in 0.0.3: ”MHM-2010 / MSM-2010 Hydrogen Maser User Guide, Rev. J” — not cited anywhere in the package, and the revision could not be verified.